

MINISTRY OF EDUCATION, CULTURE AND RESEARCH OF REPUBLIC OF MOLDOVA
TECHNICAL UNIVERSITY OF MOLDOVA
FACULTY OF COMPUTERS, INFORMATICS AND MICROELECTRONICS
DEPARTMENT OF SOFTWARE ENGINEERING AND AUTOMATICS

GG-Code: A Domain specific language for Generating and Editing G-code easier

Project report

Mentor: lect. univ., Irina Cojuhari

Students: Andrei Cliucovschi, FAF-241
Sofia Zhuchikova, FAF-241
Alexei Luchiciov, FAF-241
Dragoş Nedelea, FAF-241

Chişinău, 2026

Abstract

The project entitled “GG-Code: A Domain specific language for Generating and Editing G-code easier” was developed by students from the Technical University of Moldova. This project report consists of the domain analysis, problem definition, proposed solution architecture, and system requirements, followed by conclusions and bibliography.

Keywords: CNC, G-code, Domain-Specific Language (DSL), Digital Manufacturing, Automation, Compiler Design.

This report presents the research and initial design for a Domain-Specific Language (DSL) aimed at revolutionizing the workflow for Computer Numerical Control (CNC) and 3D printing systems. In the current digital manufacturing landscape, there exists a significant disconnect between high-level design intent and the low-level, verbose G-code instructions used to control machinery. Users are often forced to rely on opaque “black box” CAM software or resort to error-prone manual editing of cryptic machine codes.

The purpose of this project is to bridge this gap by introducing a human-readable, intermediate modelling language. This DSL allows manufacturing engineers, researchers, and hobbyists to define machine operations using programmable constructs such as variables, loops, and modular functions. The general objective is to develop a language specification and a compiler that translates these high-level descriptions into safe, optimized, and standard G-code.

The methodology employed involves a comprehensive domain analysis of current manufacturing toolchains, a Problem-Based Learning (PBL) assessment of user pain points, and a comparative analysis of existing solutions. The obtained results from this initial phase confirm the critical need for an abstraction layer in manufacturing and establish a robust set of functional requirements for the proposed system, laying the groundwork for the subsequent implementation phase.

Content

Introduction	4
1 Domain Analysis and Problem Definition	6
1.1 Domain Analysis	6
1.2 Problem Description	8
1.3 Problem Analysis	10
1.4 Problem Statement	11
1.5 Proposed Solution	11
1.6 Target Users and Customer Validation	13
1.7 Comparative Analysis	14
1.8 Problem Situation	15
1.9 Initial DSL Requirements	16
1.10 Constraints and Assumptions	16
2 Grammar Specification	18
2.1 Language Structure	18
2.2 Statement Families	18
2.3 Variable Definitions	18
2.4 Expression Evaluation	19
3 Parser Implementation	21
3.1 Design Strategy and Tooling	21
3.2 Parser Architecture	21
3.3 Implemented Language Features	22
Conclusions	24
Bibliography	25

Introduction

The field of digital manufacturing, encompassing technologies such as CNC machining and 3D printing (additive manufacturing), has become a cornerstone of modern industrial production and prototyping. The ability to translate a digital design into a physical object with micron-level precision is fundamental to sectors ranging from aerospace engineering to consumer electronics. However, despite the exponential growth in hardware capabilities, the software languages used to control these machines have remained largely static for decades. The reliance on G-code (RS-274), a low-level machine instruction language developed in the 1950s, presents a significant bottleneck in today’s increasingly agile and customized manufacturing workflows.

The motivation for this project stems from the inherent limitations of the current G-code standard. While robust and universally supported, G-code lacks the structured programming features found in modern software development, such as variables, loops, conditional logic, and meaningful variable names. This deficiency forces users to rely exclusively on complex Computer-Aided Manufacturing (CAM) software to generate code, effectively treating the machine instructions as a “black box.” When modifications, optimizations, or experimental toolpaths are required, users face a stark choice: struggle with the opaque, error-prone editing of thousands of lines of cryptic code, or accept the limitations of their CAM software.

This project addresses a critical gap in the domain: the absence of an accessible, intermediate modelling language that sits between high-level design software and low-level machine execution. The proposed solution is a Domain-Specific Language (DSL) specifically designed for describing manufacturing operations. This DSL aims to bring the power of software engineering—modularity, reusability, and abstraction—to the physical world of machine control. By allowing users to describe *what* they want to do (e.g., “drill hole pattern”) rather than *how* the motors should move, the project seeks to democratize access to advanced machine control and improve the safety and efficiency of manufacturing workflows.

The general objectives of this project are twofold. First, to perform a rigorous analysis of the problem domain, identifying the specific pain points experienced by machine operators, engineers, and researchers. Second, to design the syntax and semantics of a DSL that addresses these issues and to develop a prototype compiler capable of translating this high-level language into standard, optimized G-code.

The research methodology follows a structured Problem-Based Learning (PBL) approach. The initial phase focused on a detailed domain analysis to understand the intricacies of G-code and the limitations of existing toolchains. This involved studying the technical specifications of various machine controllers, analyzing common workflow failures, and validating the problem with potential user groups.

The structure of this report is organized as follows. **Chapter 1: Domain Analysis and Problem**

Definition provides an in-depth exploration of the manufacturing domain, a detailed breakdown of the problems associated with G-code, a comparative analysis of existing methods, and the formal definition of the system requirements and constraints. **Chapter 2: Grammar Specification** details the formal language design, syntax rules, and the derivation structures of the GG-Code language. Finally, the **Conclusions** chapter summarizes the findings of the research and outlines the future direction of the project.

This introduction sets the stage for a detailed examination of why a new language for manufacturing, GG-Code, is not just a theoretical exercise, but a practical necessity for the future of flexible automation.

1 Domain Analysis and Problem Definition

1.1 Domain Analysis

The domain of digital manufacturing has undergone a profound transformation in recent decades, evolving from manual, operator-dependent processes to highly automated, software-driven workflows. At the heart of this domain lie Computer Numerical Control (CNC) systems, which include a wide array of devices such as 3D printers, laser cutters, milling machines, and robotic arms [1]. These machines are the fundamental enablers of modern production, allowing for the precise fabrication of physical objects from digital designs. As the demand for customized, on-demand manufacturing grows, the reliability and flexibility of these systems have become paramount [2].

Digital manufacturing environments are characterized by their reliance on a complex toolchain that translates high-level design concepts into low-level machine instructions. This translation process is critical; any error or inefficiency introduced at this stage directly translates to physical defects, material waste, or machine damage. The “maker movement” and the proliferation of fabrication laboratories (FabLabs) have further democratized access to these technologies, bringing industrial-grade capabilities to hobbyists and small businesses [3, 4]. However, this democratization has also highlighted significant usability gaps in the underlying control mechanisms of these machines.

1.1.1 Machine Control Languages

The primary mechanism for controlling automated production equipment is through machine control languages, of which G-code (RS-274) is the most ubiquitous standard. Originally developed in the 1950s at the Massachusetts Institute of Technology, G-code has become the de facto language for CNC machining and, more recently, additive manufacturing processes like 3D printing [5].

G-code operates as a sequence of imperative instructions that command the machine’s hardware directly. These instructions typically fall into several categories. **Motion Commands (G-words)** control the movement of the tool or the build platform. Common examples include G0 for rapid positioning, G1 for linear interpolation at a specified feed rate, and G2/G3 for clockwise and counter-clockwise circular interpolation. **Machine Configuration (M-words)** manage auxiliary functions of the machine, such as turning the spindle on or off (M3/M5), controlling coolant flow, or managing tool changes (M6). Additionally, **Coordinate Systems** commands are used to set work offsets (e.g., G54) and switch between absolute (G90) and relative (G91) positioning modes. Finally, **Parameters** define values such as feed rate (F), spindle speed (S), and tool selection (T).

Despite its longevity, G-code is fragmented into numerous machine-specific dialects. A 3D printer running Marlin firmware may interpret specific codes differently than a Haas CNC milling machine or

a laser cutter running GRBL. This lack of standardization necessitates specific post-processors for every machine, creating a rigid coupling between the software toolchain and the hardware [6, 7].

1.1.2 Characteristics of G-code

The design of G-code reflects the computational limitations of the era in which it was conceived. It is characterized by a strictly sequential execution model, lack of high-level control flow structures (such as loops or conditionals in standard implementations), and minimal abstraction.

Low-Level Verbosity

G-code is extremely verbose. A simple operation, such as cutting a circle, might be represented by a single command or approximated by hundreds of small linear segments, depending on the post-processor. This verbosity makes G-code files difficult for humans to parse. For example, a 3D print of a moderately complex object can generate hundreds of thousands of lines of code [8].

Machine-Centricity

The language is designed for machines, not humans. Coordinates are often just numbers without semantic meaning to a reader. There are no variable names in standard G-code (though some macro dialects support them), and comments are often stripped during generation to save memory on older controllers. This opacity means that understanding *what* a specific block of code does often requires simulating the machine's state mentally or using external visualization software [9].

Lack of Abstraction

Perhaps the most significant limitation is the lack of abstraction. There is no native concept of “Drill Hole” or “Print Layer” in G-code; there is only “Move to X,Y”, “Lower Z”, “Move Z”, “Raise Z”. This absence of semantic intent means that once G-code is generated, the original design intent is lost. Modifying the code requires reverse-engineering the geometry from the motion vectors [10].

1.1.3 Current Manufacturing Software Toolchain

The standard workflow for digital manufacturing follows a linear path: CAD (Computer-Aided Design) → CAM (Computer-Aided Manufacturing) / Slicing → G-code → Machine.

In the **CAD Phase**, an engineer creates a distinct 3D geometry using parameterized modelling tools (e.g., SolidWorks, Fusion 360). This is followed by the **CAM/Slicing Phase**, where the geometry is imported into CAM software (for subtractive manufacturing) or a Slicer (for additive manufacturing). Here, the user selects tools, materials, and process parameters, and the software calculates the toolpaths required to produce the part [11, 12]. During **Post-Processing**, the calculated toolpaths are translated by a post-processor into the specific G-code dialect of the target machine. Finally, in the **Execution** phase, the G-code file is transferred to the machine and executed.

While effective for standard parts, this workflow is strictly one-way. If an error is discovered in the G-code (e.g., a movement that causes a collision but wasn't caught by the simulator), the user must

return to the CAM software, adjust parameters, regenerate the toolpath, and re-export the code. There is no easy way to “tweak” the behaviour at the machine level effectively and safely. Furthermore, different slicers can produce significantly different G-code for the same geometry, leading to inconsistencies in part quality [13, 14].

1.1.4 Domain-Specific Languages in Engineering

Domain-Specific Languages (DSLs) offer a powerful alternative to general-purpose languages for specialized problem domains. In engineering, DSLs allow experts to express solutions using concepts and terms familiar to their field, rather than low-level implementation details.

A DSL for manufacturing would bridge the gap between the high-level design intent and the low-level machine instructions. Instead of describing *how* to move a stepper motor (G-code), a user could describe the *operation* to be performed (e.g., `face_milling(depth=5mm)`). This raises the abstraction level, making the “code” readable, modifiable, and verifiable. DSLs can enforce domain rules (e.g., “spindle must be on before moving into material”) at compile time, preventing costly accidents that raw G-code would happily execute [15].

The introduction of such an intermediate language is motivated by the need for greater transparency, flexibility, and safety in modern digital manufacturing. As machines become more capable and manufacturing processes more complex, the limitations of the antiquated G-code standard become an increasing bottleneck [5, 16].

In summary, the domain analysis reveals a critical disconnect between the sophisticated capabilities of modern hardware and the primitive, rigid nature of the language used to control them.

1.2 Problem Description

The core problem addressed by this project is the lack of a human-centric, flexible, and robust method for defining and modifying manufacturing machine operations. Despite the advancements in CAD/CAM software, the final link in the chain—the machine instruction set—remains a “black box” for many users.

1.2.1 Who Experiences the Problem?

This problem profoundly affects a diverse range of practitioners within the manufacturing ecosystem. **Manufacturing Engineers and Technicians** are responsible for the efficient operation of CNC machinery. When a generated toolpath is suboptimal—for example, containing unnecessary “air cuts” or inefficient travel moves—they often lack a quick, safe way to clear non-productive movements without regenerating the entire project from the CAM software [17]. **Machine Operators** on the shop floor need to make minor adjustments to account for tool wear, material variations, or fixture offsets. Editing raw G-code manually is risky; a misplaced decimal point can crash a machine worth hundreds of thousands of dollars.

Furthermore, **Fabrication Lab (FabLab) Managers and Makers** frequently encounter artifacts in “slicing” for 3D printing that need correction. Advanced hobbyists often want to insert custom commands

(e.g., pausing for a magnet insertion, changing temperature at a specific layer) but find G-code editing cumbersome and error-prone [2]. Finally, **Research Engineers** developing new manufacturing techniques (e.g., non-planar 3D printing, hybrid additive-subtractive processes) find standard CAM software limiting because it assumes conventional workflows. They need a way to program novel machine behaviors that existing software cannot generate [18].

1.2.2 Why is this a Significant Problem?

The significance of this problem stems from the inefficiency and risk inherent in the current workflow.

First, G-code is inherently not human-friendly. Reading G-code is akin to reading assembly language, requiring significant mental effort to map coordinate values to physical space. This opacity hinders debugging; if a machine behaves unexpectedly, identifying the culprit line among thousands is essentially a needle-in-a-haystack problem [6].

Second, manual editing is error-prone. Modifications are often text-based search-and-replace operations, which are fragile and risky. Modifying a feed rate globally might accidentally affect rapid travel moves if not done carefully. Crucially, the lack of semantic validation means the machine will attempt to execute physically impossible or dangerous commands if they are merely syntactically valid G-code [19].

Furthermore, while CAM tools are powerful, they often hide the logic used to generate paths. Users who need to optimize a specific segment of the process are often forced to fight the software's defaults, finding it easier to wish they could simply write the code themselves—if only the code were writable.

Finally, in scenarios like mass customization or distributed manufacturing, there is often a need to generate variations of a part programmatically. Generating G-code directly from scripts (e.g., Python) is common but lacks structure and validation, leading to unmaintainable generators that are hard to debug.

1.2.3 Stakeholders

The key stakeholders include **Primary Stakeholders** such as machine operators, manufacturing engineers, and CNC programmers, as well as **Secondary Stakeholders** like production managers (concerned with downtime and efficiency), software developers (building manufacturing tools), and hardware vendors (interested in better ecosystem integration).

1.2.4 Relevance in the Current Landscape

The problem is becoming increasingly relevant due to the explosion of digital manufacturing. The global market for 3D printers and CNC machines is expanding rapidly [1]. As more non-experts enter the field, the barrier to entry presented by cryptic machine codes becomes a significant hurdle. Furthermore, security concerns regarding G-code manipulation and the need for validation frameworks highlight the necessity for a more structured approach to machine control [20, 19].

1.3 Problem Analysis

To fully understand the necessity for a new solution, we must analyze the underlying technical causes of the difficulties associated with G-code and the impact these issues have on the manufacturing domain.

1.3.1 Technical Causes

The root causes of the problem are deeply embedded in the legacy architecture of numerical control systems.

Low-Level Structure and Lack of Context

The fundamental issue is that G-code represents *state changes* rather than *intent*. A command like G1 X10 Y10 E0.5 tells a 3D printer to move and extrude. It does not say *why*. Is this an outer wall? An infill pattern? A support structure? Because the context is lost during generation, any subsequent modification or optimization must infer distinct features from raw coordinates. This makes “intelligent” editing impossible at the code level [10].

Machine-Specific Dialect Variations

The fragmentation of G-code dialects means that a script written for one machine is rarely portable to another without modification. This violates the principle of “write once, run anywhere” that is standard in modern software development. The variability lies not just in specific codes, but in how parameters are interpreted (e.g., relative vs. absolute arc centers), leading to subtle and dangerous bugs [21].

Complex Motion Planning Requirements

Modern machines require complex acceleration and deceleration profiles (look-ahead) to move smoothly. While the controller handles the dynamics, the G-code must provide the trajectory. Manually calculating tangent points for smooth circular transitions or calculating extrusion volumes for 3D printing is mathematically intensive and impractical for humans, enforcing reliance on “black box” CAM software.

1.3.2 Domain Impact

These technical limitations manifest as tangible inefficiencies in the manufacturing workflow.

First, workflows and iteration cycles become significantly slower. The inability to quickly edit instructions means that even minor changes require a complete round-trip to the CAD/CAM station. In a prototyping environment, this latency accumulates, slowing down the iteration cycle considerably.

Second, high barriers to entry exist for students and hobbyists. The learning curve for diagnosing machine failures is steep, as understanding why a printer failed often requires deciphering the arcane syntax of G-code.

Finally, the probability of human error increases. The cognitive load required to interpret raw G-code leads to fatigue and oversight. In industrial settings, crashes caused by incorrect code can lead to

extended downtime and expensive repairs.

1.3.3 Scenario: The Modification Bottleneck

Consider a realistic scenario involving a manufacturing engineer, Alice, who is setting up a CNC milling operation for a new aluminium housing. She generates the G-code using a high-end CAM package.

Upon running the first air-cut (a test run without material), she notices that the tool retracts to a safe height of 50mm between every single operation. For this specific fixture, a retraction to 10mm would be perfectly safe and would save 5 minutes of cycle time per part. Over a production run of 1000 parts, this saves over 80 hours of machine time.

The Problem: Alice cannot simply “change the retraction height” in the G-code file easily. 1. She opens the G-code file, which is 50,000 lines long. 2. She searches for “Z50”. It appears hundreds of times. 3. She cannot simply doing a “Find and Replace All” because “Z50” might typically be used in the setup sequence or the end sequence where it *is* required for safety. 4. She must manually verify which “Z50” commands are the inter-operation retracts and which are the safe-home retracts. 5. Alternatively, she goes back to the CAM software. She has to find the specific “Retract Height” parameter, which might be buried in a “feeds and speeds” submenu or defined globally in the machine definition. 6. She changes it, regenerates the code, and then has to re-transfer the file to the machine controller.

This process is frustratingly inefficient. If Alice had a readable, structured representation of the process, she could have simply modified a variable like `safe_retract_height = 10mm` and re-compiled, confident that the change would propagate correctly to the relevant operational blocks.

Summary: The current state of affairs forces a trade-off between the safety of trusted but rigid CAM generation and the flexibility but danger of manual G-code editing.

1.4 Problem Statement

Manufacturing practitioners lack an accessible, programmable, and expressive method for modelling and modifying machine operations because existing workflows rely wholly on either complex, opaque CAM systems or low-level, difficult-to-interpret G-code instructions.

Elaboration: This statement highlights the gap in the current ecosystem. “Accessible” refers to the ease of use for non-computer-scientists. “Programmable” indicates the need for logic (loops, variables) which static G-code lacks. “Expressive” implies the ability to capture intent (e.g., “drill hole”) rather than just movement. The problem is framed as a dichotomy between high-level tools that offer little control over the output details and low-level output that offers total control but zero usability. The proposed DSL seeks to occupy the middle ground.

1.5 Proposed Solution

To address the limitations of G-code and the rigidity of standard CAM workflows, this project proposes the development of the ****GG-Code DSL (Domain-Specific Language)****. This language will

serve as an intermediate abstraction layer between user intent and machine execution.

1.5.1 Conceptual Architecture

The proposed system operates on a compilation model similar to software development languages. The process begins with the **Source Model (DSL)**, where the user defines the manufacturing process using high-level commands, variable definitions, and control structures. This file is human-readable and semantically rich. Next, the **Parser and Validator** reads the source file, parsing the syntax and performing semantic validation. It checks for errors such as undefined variables, out-of-bounds parameters, or logical inconsistencies. The valid model is then transformed through **Translation/compilation** into an abstract syntax tree (AST) and processed to generate the target machine instructions. Finally, the **Output (G-code)** is emitted as standard, optimized G-code compatible with the specific target machine (e.g., Marlin, GRBL, Fanuc).

This architecture decouples the description of the *operation* from the implementation of the *instruction*.

1.5.2 Advantages of the DSL Approach

Higher Readability

The DSL uses English-like syntax and meaningful keywords. Instead of `G0 Z5.0`, the code might read `MoveTo(Z=SafeHeight)`. This self-documenting nature makes it easier for teams to collaborate on manufacturing scripts and for new users to understand existing processes [15].

Reusable Abstractions

Users can define functions or macros for common tasks. If a specific cleaning wiping motion is required before every print, it can be defined once as a function `WipeNozzle()` and called repeatedly. In standard G-code, this sequence of movements would have to be copy-pasted every time, leading to bloated and unmaintainable files.

Easier Editing of Machine Behaviour

Modifying process parameters becomes trivial. Changing the feed rate for a specific type of movement requires changing one variable definition at the top of the file. The compiler ensures this change propagates consistently throughout the generated code [10].

Safer Workflow

The compiler acts as a safety barrier. It can enforce constraints such as maximum feed rates, travel limits, or forbidden zones. It can warn the user if a generated path would exceed the machine's build volume, catching errors *before* the machine starts moving [16].

1.5.3 Example Scenario: User Interaction

Imagine a researcher, Bob, testing a new type of material deposit strategy that requires a zigzag pattern with varying speed.

Without DSL: Bob tries to draw the zigzag in CAD, but setting the variable speed is impossible

in standard CAM. He resorts to writing a Python script to print strings of “G1 X...F...” commands. His script is buggy, 200 lines long, and hard to debug.

With DSL: Bob writes a short DSL script:

```
speed_fast = 3000
speed_slow = 500
y_pos = 0

repeat(10) {
    Move(x=0, y=y_pos, speed=speed_fast)
    y_pos = y_pos + 1
    Move(x=100, y=y_pos, speed=speed_slow)
    y_pos = y_pos + 1
}
```

The DSL compiler takes this concise looping construct and unrolls it into the correct sequence of G-code coordinates. Bob can easily change the number of repetitions or the speeds and re-generate the file in seconds. This example illustrates how the DSL empowers users to express algorithmic intent that is cumbersome to define in both CAD and raw G-code.

1.5.4 Summary

The proposed solution moves the complexity from the user’s manual effort to the compiler’s logic. By treating manufacturing instructions as software code, we can apply software engineering best practices—modularity, reuse, validation—to the physical world of manufacturing.

1.6 Target Users and Customer Validation

The success of the DSL depends on its adoption by key user groups who currently face friction in the manufacturing workflow.

1.6.1 Target User Groups

Manufacturing Engineers constitute the primary user base for industrial applications. They require fine-grained control over machine operations to optimize cycle times and ensure quality. A DSL allows them to build libraries of optimized routines for specific materials or parts, creating a knowledge base that is currently lost in disconnected G-code files. **Machine Operators and Technicians** often need to make “quick fixes” on the shop floor. A readable DSL allows them to understand what a program is intended to do without needing to act as a human G-code parser. It empowers them to make safe adjustments (e.g., speed overrides, offset changes) within a controlled framework.

The **Maker community** is a significant driver of 3D printing innovation. Hobbyists frequently experiment with non-standard modifications to their machines (e.g., plotting with pens, paste extrusion).

A DSL provides them with an accessible tool to program these custom tools without writing full-blown firmware or post-processors [2]. Similarly, **Research Engineers** in academic and industrial settings experimenting with new fabrication processes (e.g., 4D printing, functionally graded materials) need to step outside the bounds of commercial CAM software. The DSL provides a flexible platform for defining novel toolpath strategies that standard slicers simply cannot generate [18].

1.6.2 Customer Validation

Validation logic suggests that these users are actively seeking alternatives. The prevalence of custom Python scripts, spreadsheets for G-code calculation, and forum threads asking “how to edit G-code” serves as validation of the unmet need. By providing a structured, robust tool, the DSL addresses a clear pain point in the market.

1.7 Comparative Analysis

To situate the proposed solution within the existing technology landscape, we compare it against the two dominant approaches: Raw G-code programming and CAD/CAM Systems.

1.7.1 Approach 1: Raw G-code Programming

Writing commands directly in a text editor offers ultimate control, making every byte of the file explicit without “hidden” behaviors inserted by software. It also requires no expensive software licenses. However, it is extremely tedious and error-prone. Calculating coordinates for complex geometries (e.g., a splined curve) is practically impossible for a human. It lacks safety checks; a typo can cause physical damage. Debugging is difficult due to low readability [6].

1.7.2 Approach 2: CAD/CAM Systems

Using graphical software to model parts and generate toolpaths automatically is highly efficient for complex geometry, and visual simulation prevents many errors. It is the industry standard for part production [11]. Disadvantages include acting as a “black box,” where users have limited control over the specific G-code output. Customizing the output requires editing complex “post-processor” files, which is disjointed from the user interface. It is often expensive and resource-heavy [12].

1.7.3 Approach 3: Proposed DSL

An intermediate textual language that compiles to G-code combines the precision of coding with the abstraction of high-level software. It allows for “algorithmic usage” (loops, logic) which CAM lacks and G-code handles poorly. It produces standard G-code, maintaining compatibility with existing machines. It is text-based, making it version-control friendly (Git). However, it requires users to learn a new syntax and is not suitable for designing organic 3D shapes (sculpting), which is better left to CAD. It represents a shift in mindset from “drawing” to “programming” manufacturing.

Conclusion: The DSL does not replace CAD/CAM for complex geometry but complements it by handling algorithmic, repetitive, or highly customized machine operations where standard CAM fails. It

replaces raw G-code editing entirely [21].

1.8 Problem Situation

The manufacturing industry relies heavily on Computer Numerical Control (CNC) machines to interpret digital designs into physical products. Currently, the industry standard for communicating with these machines is G-code, a language developed in the 1950s. While robust, G-code is cryptic, unstructured, and difficult for humans to read or modify safely.

Modern workflows involve generating this code via CAD/CAM software, which effectively hides the code from the user. However, when engineers, researchers, or operators need to customize machine behavior, debug errors, or optimize specific movements, they are forced to interact directly with this user-unfriendly low-level code. There is no intermediate tool that allows for high-level, programmable control of the machine without the overhead of full CAD/CAM systems or the risk of manual text editing.

The absence of such a tool leads to inefficiencies, increased risk of machine damage, and a barrier to innovation in experimental manufacturing processes. Users resort to ad-hoc scripts or manual hacking of files, which is sustainable for neither industrial production nor advanced research.

1.8.1 Problem Sentence

There is a lack of an intermediate, expressive modelling language that allows manufacturing professionals to program and modify machine operations efficiently, bridging the gap between rigid CAD/CAM generators and low-level G-code instructions.

1.8.2 Structured Analysis

The analysis can be broken down into specific components. **Who:** Manufacturing engineers, CNC operators, 3D printing specialists, researchers, and makers. **What:** The difficulty in creating, reading, and verifying machine instructions (G-code) due to its low-level nature and lack of abstraction. **Where:** In fabrication laboratories, extensive manufacturing plants, prototyping workshops, and research facilities globally. **When:** During the “post-processing” phase of manufacturing, immediately before production, or during the optimization of repetitive production cycles. **Why:** Because G-code is archaic and verbose, while CAD/CAM is opaque and inflexible. An intermediate layer is needed to provide control and safety. **How:** By developing a Domain-Specific Language (DSL) and a compiler that translates high-level intents into valid machine code. **How Much:** The scale affects the entire digital manufacturing industry. Reducing setup time by even small percentages can save millions in industrial contexts; enabling research can lead to new manufacturing paradigms.

1.8.3 Deliverable

The project will deliver a **DSL Language Specification** defining the syntax and semantics for machine operations, and a **Prototype Transpiler/Compiler** capable of converting DSL source files into valid, machine-readable G-code.

1.8.4 Conclusion

The PBL analysis confirms that the problem is well-defined, impacts a significant user base, and has a clear technological solution. The development of the DSL addresses the root causes of opacity and rigidity in the current workflow.

1.9 Initial DSL Requirements

To effectively solve the identified problem, the DSL must meet the following functional requirements.

1.9.1 Motion and Geometry

The DSL must support the core requirement of describing movement in physical space. This includes **Linear Motion** for precise control over straight-line movements (G0, G1) in 3D space (X, Y, Z), and **Arc/Circular Motion** to define curves and circles (G2, G3) intuitively, potentially abstracting away the complex center-point calculations required by raw G-code. Furthermore, **Coordinate Systems** support is essential for absolute and relative positioning, allowing users to define movements relative to the current position (useful for patterns) or relative to the machine origin [9].

1.9.2 Machine Operations

Beyond movement, the language must control the machine’s state. This encompasses **Tool Control** commands to start/stop spindles, control laser power, or manage extruder temperatures, as well as **Tool Changes** via high-level commands (e.g., ‘SwitchTool(T1)’ which the compiler translates into the safe sequence of retraction, changes, and priming. It must also support **Wait/Pause** instructions to dwell or pause for operator intervention.

1.9.3 Abstraction and Control Flow

These are the key differentiators from G-code. **Variables and Parameters** allow users to define values (e.g., `layer_height`, `travel_speed`) and use them throughout the script. **Loops and Repetition** constructs enable repeating operations (e.g., for creating layers or grid patterns) without copy-pasting code. Finally, **Modular definitions** provide the ability to define reusable blocks or functions (macros) for common geometry or setup sequences [5].

1.9.4 Code Generation

The system must ensure **Validity**, meaning the output must be syntactically valid G-code. It must also ensure **Compatibility**, supporting configuration for different G-code “flavors” (e.g., Marlin vs. standard NIST G-code).

1.10 Constraints and Assumptions

The project is bounded by specific constraints and relies on certain assumptions to be feasible.

1.10.1 Constraints

The project must adhere to **G-code Standard Compatibility**, ensuring the generated output adheres to the RS-274 standard (or common industry variations), as we cannot invent a new protocol that requires firmware changes on the machines. **Safety** is paramount; the DSL compiler must not generate instructions that are inherently dangerous (e.g., negative extrusion, movement beyond physical limits if dimensions are known). Finally, **Determinism** dictates that the compilation process must be deterministic, where the same DSL input always produces the exact same G-code output.

1.10.2 Assumptions

We assume **User Knowledge**, meaning the target user understands the basic principles of CNC machining (e.g., coordinate systems, feed rates); the DSL simplifies the *coding*, not the *physics*. We also assume a **Preprocessing Layer** architecture where the DSL functions as a pre-processor running on a computer to generate a file sent to the machine, rather than a real-time interpreter on the microcontroller. Finally, we assume **Target Machine Capabilities** include the execution of standard G-code commands.

2 Grammar Specification

This chapter details the formal grammar design of the GG-Code language. The language is designed to be deterministic, unambiguous, and readable, drawing inspiration from natural language commands while maintaining strict syntactic structure. The grammar is defined using a recursive descent approach, with clear separation between statement types and expression evaluation.

2.1 Language Structure

The top-level structure of a GG-Code program consists of a sequence of statements. The parser processes these statements sequentially, allowing for a straightforward execution model. Unlike free-form natural language processing, the grammar enforces a rigid structure where every command must begin with a reserved keyword, ensuring predictability and simplifying the compilation process.

2.2 Statement Families

Statements in GG-Code are categorized into five distinct families: motion, machine control, geometry creation, variable definition, and control flow. This categorization aids in modularizing the parser and semantic analyzer.

2.2.1 Motion Commands

Motion commands control the physical movement of the machine tool. The primary command is `Move`, which accepts target coordinates specified either by axis values (e.g., `X10 Y20`) or by named locations. The grammar also supports stopping and pausing operations, which are critical for multi-stage manufacturing processes.

2.2.2 Control Flow Structures

To support algorithmic design, the language includes robust control flow mechanisms. The `Repeat` loop allows for fixed-count iterations, essential for layered manufacturing. Conditional logic is handled via `If . . . Then . . . Else` constructs, enabling the machine to make decisions based on runtime variables or sensor data (simulated).

Figure 2.1 illustrates the syntactic structure of a loop, showing how the `RepeatStatement` node encapsulates the count expression and the block of statements to be executed.

Similarly, Figure 2.2 depicts the conditional structure. The grammar ensures that the condition expression is evaluated before executing the `Then` block or the optional `Else` block.

2.3 Variable Definitions

Variables and named locations provide the necessary abstraction to make the code reusable. The `Define` statement binds a meaningful name to a specific coordinate or location, while the `Set` statement assigns values to variables used in expressions.

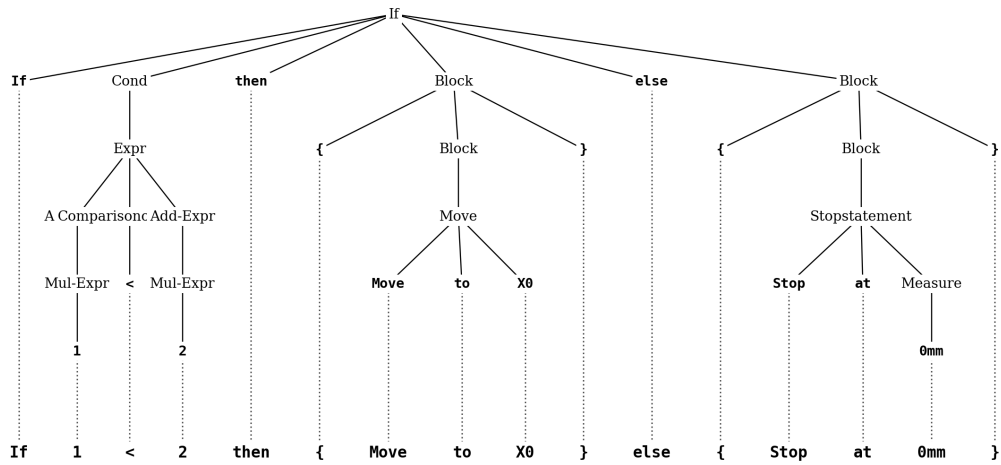


Figure 2.2 - Derivation Tree for a Conditional Statement

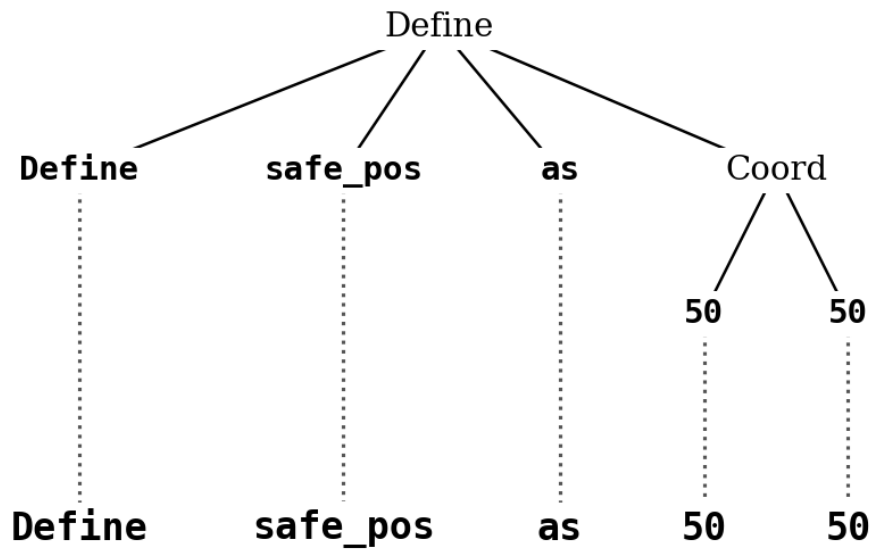


Figure 2.1 - Derivation Tree for a Definition Statement

3 Parser Implementation

This chapter describes the implementation strategy for the GG-Code parser, detailing the architectural choices made to ensure the system meets the requirements of determinism, extensibility, and robust error handling identified in the domain analysis.

3.1 Design Strategy and Tooling

Based on the complexity of the grammar defined in the previous chapter, a decision was made to utilize a specialized parser generator rather than implementing a hand-written recursive descent parser. This choice was driven by several key factors:

1. **Maintainability:** A formal grammar file serves as a single source of truth for the language syntax. Changes to the language structure can be implemented by modifying the grammar rules rather than rewriting complex parsing logic code.
2. **Robustness:** Generated parsers provide proven, well-tested algorithms for handling edge cases in tokenization and lookahead, reducing the likelihood of parser bugs compared to custom implementation.
3. **Visualization:** The ability to automatically generate parse trees (as seen in Chapter 2) is invaluable for debugging and visualizing the structure of user code.

The implementation utilizes an LL(*) parsing algorithm. This top-down approach is particularly suitable for our command-oriented language because the initial keywords (e.g., Move, Repeat, If) provide strong predictive cues, allowing the parser to efficiently select the correct rule path without excessive backtracking.

3.2 Parser Architecture

The parsing subsystem is divided into three distinct stages: Lexical Analysis, Syntactic Analysis, and Abstract Syntax Tree (AST) Construction.

3.2.1 Lexical Analysis (Tokenization)

The first stage involves breaking the raw input text into a stream of meaningful tokens. The lexer is configured to recognize:

- **Keywords:** Reserved command words (e.g., Move, Pause) are identified case-insensitively to improve user experience, though they are standardized internally.
- **Literals:** Numeric values, including integers and decimals, are parsed alongside specific unit tokens (e.g., mm, C) to ensure measurements are correctly engaged.
- **Identifiers:** User-defined names for variables and locations are distinguished from reserved keywords.
- **Whitespace and Comments:** Non-significant whitespace and comments are discarded at this stage to simplify the downstream parsing logic.

3.2.2 Syntactic Analysis

The parser consumes the token stream and verifies that it conforms to the grammar rules. It builds a concrete parse tree where every node represents a grammatical rule application. If the input violates the grammar (e.g., missing a parameter or incorrect nesting), the parser halts and reports a syntax error with line and column information, satisfying the requirement for immediate user feedback.

3.2.3 AST Construction

While the concrete parse tree perfectly reflects the grammar, it is often too verbose for code generation. The final stage of the parsing functionality is the transformation of this parse tree into an Abstract Syntax Tree (AST).

The AST builder traverses the parse tree (using the Visitor pattern) and simplifies the structure into semantically meaningful objects.

- Structural tokens (punctuation like curly braces, commas) are discarded.
- Implicit relationships in the grammar are made explicit in the AST nodes (e.g., a `RepeatStatement` node directly contains a count and a body, rather than a list of generic child nodes).
- Flattening of expression chains occurred, so that `'1 + 2 * 3'` is represented as a tree of operations respecting precedence.

This constructed AST serves as the clean, validated input for the subsequent Semantic Analysis and Code Generation phases, ensuring that the compiler backend operates on "valid-by-construction" data structures.

3.3 Implemented Language Features

This section serves as a technical reference for the functionality currently supported by the GG-Code parser. The language is designed to cover the core requirements of manufacturing while providing the necessary abstractions for algorithmic design.

3.3.1 Motion Control

Precision movement is the foundation of G-code generation. The parser supports the following motion commands:

- **Move To:** A versatile command `Move to <target>` that handles various target formats.
 - *Coordinates:* Direct axis values (e.g., `X10, Y20, Z5.5`).
 - *Predefined Locations:* Reserved keywords `Base`, `Origin`, and `Center Point`.
 - *User-Defined Locations:* Named targets defined earlier in the script (e.g., `Move to start_pos`).
- **Stop/Pause:** `Stop at <measure>` for halting at specific heights (e.g., `20mm`) and `Pause at Layer <int>` for layer-based interventions.
- **Home:** Use `Home` to home all axes or `Home X Y` to home specific axes.

3.3.2 Machine Configuration

These commands manage the physical state of the machine beyond movement.

- **Temperature:** `Temperature <val>C` (e.g., `Temperature 200C`) sets the extruder or bed temperature.
- **Wait:** `Wait for <time>` allows for dwelling, supported in Seconds or Minutes.
- **Tool Change:** `Use <tool_name>` triggers tool change sequences.

3.3.3 Geometric Primitives

To raise the level of abstraction, the language supports high-level geometry definitions which the compiler expands into appropriate motion paths.

- **Square:** `Add Square Center <x> <y> Length <val>`
- **Rectangle:** `Add Rectangle Center <x> <y> Width <w> Length <l>`
- **Circle:** `Add Circle Center <x> <y> Radius <r>`
- **Line:** `Add Line From <x1> <y1> To <x2> <y2>`

3.3.4 Control Flow and Logic

Unlike standard G-code, GG-Code supports structured programming constructs.

- **Repetition:** `Repeat <n> times { ... }` allows for easy iteration of layers or patterns.
- **Conditional Execution:** `If <condition> then { ... } else { ... }` enables adaptive tool-paths based on variables.
- **While Loops:** `While <condition> { ... }` for state-dependent iteration.

3.3.5 Variables and Expressions

The parser includes a full expression engine supporting:

- **Definitions:** `Define <name> as <location>` for storing coordinates.
- **Assignments:** `Set <variable> to <expression>` for dynamic value updates.
- **Math:** Standard operators (+, -, *, /) and parentheses for grouping.
- **Comparisons:** Relational operators (==, !=, <, >, <=, >=) for logic gates.

3.3.6 Lexical Features

To ensure usability, the following lexical features are implemented:

- **Case Insensitivity:** Keywords such as `MOVE` and `move` are treated identically.
- **Units:** Explicit support for `mm`, `cm`, `C` (Celsius) attached to literals.
- **Comments:** C-style line comments (`//`) and block comments (`/* ... */`) are supported and stripped during tokenization.

Conclusions

The detailed analysis conducted in the first phase of this project has confirmed that while the digital manufacturing industry has advanced significantly in terms of hardware, the software infrastructure for machine control remains rooted in the mid-20th century. The reliance on G-code as the primary interface for Computer Numerical Control (CNC) and additive manufacturing systems creates a significant barrier to efficiency, customization, and safety.

The Problem-Based Learning (PBL) investigation highlighted three critical issues: the lack of readability and modifiability of machine instructions, the absence of abstraction which hides design intent, and the steep learning curve required to debug or optimize manufacturing processes manually. These issues affect a wide spectrum of users, from hobbyists in fabrication laboratories to engineers in industrial settings.

The proposed solution—a Domain-Specific Language (DSL)—emerges as a necessary and viable innovation. By introducing high-level programming concepts such as variables, loops, functions, and semantic operations into the manufacturing workflow, the DSL promises to transform machine instructions from cryptic data streams into readable, maintainable software code. This approach not only simplifies the creation of complex toolpaths but also introduces a compilation step where safety checks and optimizations can be performed automatically, reducing the risk of costly machine failures.

The comparative analysis demonstrates that while raw G-code offers control and CAD/CAM offers visualization, neither provides the programmable flexibility required for modern agile manufacturing or experimental research. The proposed DSL bridges this gap, offering the best of both worlds: high-level abstraction with deterministic low-level output.

In conclusion, the domain analysis and problem definition phase has laid a solid foundation for the development of the DSL. The functional requirements and constraints identified herein provide a clear roadmap for the next phase of the project: the formal specification of the language syntax and the implementation of the prototype compiler. This tool has the potential to significantly streamline the prototyping workflow and empower users to leverage the full capabilities of their manufacturing hardware.

Bibliography

- [1] Global 3D printing industry market size. Available from: <https://www.statista.com/statistics/315386/global-market-for-3d-printers/>
- [2] Mavri, M.; Fronimaki, E.; et al. Survey analysis for the adoption of 3D printing technology: consumers' perspective. volume ahead-of-print, doi:10.1108/JSTPM-02-2020-0023.
- [3] cycles, T. t. p. g. i. S. a. n. l. f. t. i. g. b. c. o. c. D. t. v. u.; Text, S. C. D. M. u.-t.-D. D. T. R. i. t. Topic: Additive manufacturing and 3D printing. Available from: <https://www.statista.com/topics/1969/additive-manufacturing-and-3d-printing/>
- [4] Between machines and art: The impact of CNC technology on artistic creation. volume 27: p. 100466, ISSN 2590-0056, doi:10.1016/j.array.2025.100466. Available from: <https://www.sciencedirect.com/science/article/pii/S2590005625000931>
- [5] Arroyo, G.; Ramos, J.; et al. *Generating CNC Code from a Domain Specific Language*.
- [6] Tiro, D. Rapid G-Coding – Obtaining G-code Using AI. volume 1339: p. 012014, doi:10.1088/1757-899X/1339/1/012014.
- [7] The Ultimate Guide to CNC Macro Variables for Smarter Machining. Available from: <https://cncmacros.com/the-ultimate-guide-to-cnc-macro-variables-for-smarter-machining>
- [8] Beyer, M.; Burde, A.; et al. A Slicer-Independent Framework for Measuring G-Code Accuracy in Medical 3D Printing. volume 12, no. 1: p. 25, ISSN 2313-433X, doi:10.3390/jimaging12010025. Available from: <https://www.mdpi.com/2313-433X/12/1/25>
- [9] Propst, S.; Mueller, J. Time Code for multifunctional 3D printhead controls. volume 16, no. 1: p. 1035, ISSN 2041-1723, doi:10.1038/s41467-025-56140-1. Available from: <https://doi.org/10.1038/s41467-025-56140-1>
- [10] Abdelaal, M.; Lokadjaja, S.; et al. *GLLM: Self-Corrective G-Code Generation using Large Language Models with User Feedback*. doi:10.48550/arXiv.2501.17584.
- [11] From CAD to G-code: Strategies to minimizing errors in 3D printing process. volume 55: pp. 62–70, ISSN 1755-5817, doi:10.1016/j.cirpj.2024.09.005. Available from: <https://www.sciencedirect.com/science/article/pii/S175558172400141X>
- [12] How to Convert A Cad File Into G-Code? - Learn from Expert Guide. Available from: <https://www.machinestation.us/how-do-i-convert-a-cad-file-into-g-code/>
- [13] Beyer, M.; Burde, A.; et al. A Slicer-Independent Framework for Measuring G-Code Accuracy in Medical 3D Printing. volume 12: p. 25, doi:10.3390/jimaging12010025.
- [14] Bryła, J.; Martowicz, A. Study on the Importance of a Slicer Selection for the 3D Printing Process

- Parameters via the Investigation of G-Code Readings. volume 9, no. 8: p. 163, ISSN 2075-1702, doi:10.3390/machines9080163. Available from: <https://www.mdpi.com/2075-1702/9/8/163>
- [15] Zhang, Z.; Gan, Y.; et al. *Still Manual? Automated Linter Configuration via DSL-Based LLM Compilation of Coding Standards*. doi:10.48550/arXiv.2602.07783.
- [16] Auto-seg: An automated G-code interpreter and 1DCNN-based framework for signal segmentation and synchronization in CNC machining. volume 191: p. 114644, ISSN 1568-4946, doi:10.1016/j.asoc.2026.114644. Available from: <https://www.sciencedirect.com/science/article/abs/pii/S156849462600092X>
- [17] Abdeljabbar, N.; Hammadi, M. *A Digital Hybrid Pipeline for Energy-Aware CNC Machining: Surrogate Models Leveraging CAM and G-Code Data*.
- [18] Riffel, K. C.; Bawa, R.; et al. Artificial Intelligence and Statistical Mapping Applied to Additive Manufacturing Toolpath Optimization in Wire-Arc DED. volume 14, no. 3: pp. 425–441, ISSN 2193-9772, doi:10.1007/s40192-025-00415-4. Available from: <https://doi.org/10.1007/s40192-025-00415-4>
- [19] Kurkowski, E.; Van Stockum, A.; et al. MANIPULATION OF G-CODE TOOLPATH FILES IN 3D PRINTERS: ATTACKS AND MITIGATIONS. In *Critical Infrastructure Protection XVI*, volume 666, edited by J. Staggs; S. Sheno, Springer Nature Switzerland, ISBN 978-3-031-20136-3 978-3-031-20137-0, pp. 155–174, doi:10.1007/978-3-031-20137-0_6, series Title: IFIP Advances in Information and Communication Technology. Available from: https://link.springer.com/10.1007/978-3-031-20137-0_6
- [20] Machine learning-based early detection of malicious G-code manipulations in 3D printing. volume 145: pp. 211–235, ISSN 1526-6125, doi:10.1016/j.jmapro.2025.04.012. Available from: <https://www.sciencedirect.com/science/article/abs/pii/S1526612525003937>
- [21] Secrets of Going From CAD, Image, DXF, or STL to GCode for CNC and 3D Printing. Available from: <https://www.cnccookbook.com/secrets-going-cad-image-dxf-stl-gcode-cnc-3d-printing/>
- [22] Woźniak, M.; Krason, J.; et al. The Effect of 3D Printer Head Extruder Design on Dynamics and Print Quality. volume 15: p. 13140, doi:10.3390/app152413140.